PROJECT SUMMARY REPORT
Secure Login System

The project delivers a fully functional secure authentication platform integrating registration, login, RBAC (Role-Based Access Control), CAPTCHA verification, and a lockout defense mechanism. The system is designed to operate like a minimal yet robust identity layer—lean, enforceable, and built with a security-first mindset.

1. System Overview

The platform enables:

- User Registration & Login
- Role-Based Access (Admin / User)
- Protected routes enforced via JWT
- Security Layers: bcrypt hashing, login attempt throttling, CAPTCHA, token validation

Tech stack features a classic triad:

- Frontend (HTML/CSS/JS), Backend (Node.js + Express), Database (MongoDB Atlas).

2. UI & User Experience

Screenshots across pages 2 and 8 show a polished, gradient-based UI with dedicated layouts for:

- Login
- Registration
- Admin Dashboard
- CAPTCHA validation screen

The admin dashboard includes user management tables with actions for role and status—mirroring a lightweight IAM panel.

3. Frontend Implementation

- Registration form markup with username, email, password, role selection
- CSS layout tailored for centered forms, card-style containers
- JavaScript validation ensuring password checks and frontend error messaging

Frontend plays gatekeeper before requests hit the API.

4. Backend Architecture

- Server (server.js)
    - Express setup
    - Middleware for JSON parsing
    - Auth route integration
    - Port binding & environment config
- Database Layer
    - MongoDB connection via mongoose
    - User schema including username, email, password hash, role, failed attempts, timestamps

5. API Endpoints

The major pipelines:

- Registration API
    - Validates user uniqueness, encrypts password via bcrypt, and writes user profile into MongoDB.
- Login API
    - Verifies email/password pair, signs JWT with role metadata, returns token to the client.
    - Documentation clearly showcases code snippets illustrating error handling and status responses.

- RBAC Middleware
    - JWT verification followed by role authorization enforcement.
    - Requests for missing valid tokens are rejected with 401/403—exactly how a security-aligned backend behaves.

6. Security Enhancements

The project goes beyond the bare minimum:

- CAPTCHA Integration
    - Shown on page 8. This mitigates automated brute-forcing.
- Account Lockout Mechanism
    - After repeated failed attempts, system forces a 15-minute cooldown to slow credential-stuffing attacks.
- Password Hashing
    - bcrypt ensures no plaintext secrets ever touch the database.
- JWT-Based Session Layer
    - Stateless, secure, and role-aware.

7. Database & Testing

Testing verifies:

- Input validation
- Login correctness
- JWT validation
- Admin-only route behavior
- Lockout triggers
- Duplicate email handling

8. Final Outcome

According to the documentation, the completed system successfully achieves:

- Secure user authentication
- Dual-role access separation
- Functional UI for all flows
- Backend services with proper validation, hashing, CAPTCHA, and RBAC
- Admin dashboard for user oversight